logix.otf
logix.otf[Scale=1.0,NFSSFamily=logix]
"0"E79E "0"E79F "0"E667
"0"E810 "0"E811 "0"E820 "0"E821 "0"E830 "0"E831 "0"E840 "0"E841 "0"E850 "0"E851 "0"E860 "0"E861 "0"E870 "0"E871
"0"E814 "0"E815 "0"E824 "0"E825 "0"E834 "0"E835 "0"E844 "0"E845 "0"E854 "0"E855 "0"E864 "0"E865 "0"E874 "0"E875
"0"E818 "0"E819 "0"E828 "0"E829 "0"E838 "0"E839 "0"E848 "0"E849 "0"E858 "0"E859 "0"E868 "0"E869 "0"E878 "0"E879
"0"E81C "0"E81D "0"E82C "0"E82D "0"E83C "0"E83D "0"E84C "0"E84D "0"E85C "0"E85D "0"E86C "0"E86D "0"E87C "0"E87D
"0"E8B8 "0"E8B9 "0"E8BC "0"E8BD "0"E8C8 "0"E8C9 "0"E8CC "0"E8CD "0"E8D8 "0"E8D9 "0"E8DC "0"E8DD "0"E8E0 "0"E8E1
"0"E812 "0"E813 "0"E822 "0"E823 "0"E832 "0"E833 "0"E842 "0"E843 "0"E852 "0"E853 "0"E862 "0"E863 "0"E872 "0"E873
"0"E816 "0"E817 "0"E826 "0"E827 "0"E836 "0"E837 "0"E846 "0"E847 "0"E856 "0"E857 "0"E866 "0"E867 "0"E876 "0"E877
"0"E81A "0"E81B "0"E82A "0"E82B "0"E83A "0"E83B "0"E84A "0"E84B "0"E85A "0"E85B "0"E86A "0"E86B "0"E87A "0"E87B
"0"E81E "0"E81F "0"E82E "0"E82F "0"E83E "0"E83F "0"E84E "0"E84F "0"E85E "0"E85F "0"E86E "0"E86F "0"E87E "0"E87F
"0"E8BA "0"E8BB "0"E8BE "0"E8BF "0"E8CA "0"E8CB "0"E8CE "0"E8CF "0"E8DA "0"E8DB "0"E8DE "0"E8DF "0"E8E2 "0"E8E3
"0"E94C "0"E96C "0"E940 "0"E960 "0"E94E "0"E96E "0"E942 "0"E962 "0"E944 "0"E964 "0"E946 "0"E966 "0"E980 "0"E982
"0"E908 "0"E928 "0"E90A "0"E92A "0"E90C "0"E92C "0"E90E "0"E92E
"0"E602 "0"E603 "0"E606 "0"E605
"0"E718 "0"E719 "0"E71A "0"E71B
"0"E738 "0"E739 "0"E73A "0"E73B
"0"E768 "0"E769 "0"E76A "0"E76B
"0"E798 "0"E799 "0"E79A "0"E79B
"0"E678 "0"E679 "0"E67A "0"E67B
"0"E079 "0"E07A "0"E07B "0"E07C "0"E07D "0"E07E
"0"E6A8 "0"E6A9 "0"E6AA "0"E6AB "0"E656 "0"E657 "0"E658 "0"E659 "0"E6AE "0"E6AF "0"E6AC "0"E6AD
"0"E688 "0"E689 "0"E68A "0"E68B "0"E68E "0"E68F "0"E68C "0"E68D
"0"E6C8 "0"E6C9 "0"E6CA "0"E6CB "0"E6CE "0"E6CF "0"E6CC "0"E6CD

Clase 2

Recursos compartidos

1

Debe garantizarse exclusión mutua entre procesos para que dos procesos no accedan al mismo tiempo a una sección crítica (recurso compartido).

El problema del jardín ornamental consiste en un jardón con dos entradas (este, oeste) con puertas giratorias. Se desea contar el número de personas que ingresan. Se desea una variable global (compartida) para el contador.

Clase 3

Lenguajes ω-regulares

Sea Σ^* un lenguaje y $A \subseteq \Sigma^*$. Definimos

$$A^\omega := \{\sigma_1\sigma_2\sigma_3 \ldots \mid \forall i \geq 1 : \sigma_i \in A \wedge \sigma \neq \epsilon\} \tag{1}$$

Es decir, A^ω contiene todas las concatenaciones infinitas de palabras no vacías de A.

Un lenguaje L se dice ω-regular si existen lenguajes regulares A_i, B_i, con $0 \leq i \leq k$, tal que $\epsilon \notin B_i \neq \emptyset$ y

$$L = \bigcup_{i=0}^{k} A_i B_i^\omega$$

donde $A_i B_i^\omega$ es la concatenación de lenguajes.

Teorema. Los lenguajes ω-regulares son cerrados por unión, unión, intersección y complemento.

Una palabra $\alpha \in A^\omega$, con $A \subseteq \Sigma^*$, se denomina un fragmento de traza.

> **Ejemplo.** Considere el lenguaje $A = [(m+\epsilon)(r+u)]^* \subseteq \{m, r, u\}^*$. Ahora considere A^ω el conjunto de concatenaciones no vacías de A.
>
> Una palabra de A es mru. Una palabra de A^ω es
>
> $$mru\ rrr\ mu\ mru\ rrr\ mu \ldots$$
>
> Notar que si definimos $X = \{\epsilon\}$, $L = XA^\omega = A^\omega$ es un lenguaje omega regular.
>
> También podríamos definir $\Sigma := \{\text{poweron}, \text{setup}, \text{yield}\}$ y $S = \{\text{poweron} \cdot \text{setup} \cdot \text{yield}\}$ un alfabeto con una sola palabra. Entonces
>
> $$L = SA^\omega$$
>
> es un lenguaje ω-regular con «cadena de iniciación» dada por la única palabra de S y luego trazas tomadas de A^ω.
>
> En general, $L = \bigcup_{i=0}^{k} A_i B_i^\omega$ define una unión de lenguajes que tienen ciertas cadenas de inicialización seguidas de trazas infinitas (no vacías).

Los lenguajes ω-regulares sirven para escribir propiedades de sistemas concurrentes. En general, una propiedad P de un lenguaje se define como un subconjunto de trazas $P \subseteq \Sigma^\omega$.

Safety

La propiedad de safety especifica que «nunca ocurre» algo, donde el *algo* en cuestión se entiende como un evento indeseable. Una posible especificación de safety es el conjunto de trazas $P \subseteq \Sigma^\omega$ tales que

$$\sigma \notin P \Rightarrow \exists i \geq 0 : \forall \beta : \sigma[..i]\beta \notin P \tag{2}$$

donde $\sigma[..i]$ es el prefijo de σ de longitud i.

Desglosemos. Si σ es una traza que ha violado la propiedad de safety, entonces la ha violado en un instante de tiempo i determinado, y a partir de dicho momento no importa qué β siga después, la traza sigue violando la propiedad de safety. En otras palabras, la traza ha violado la propiedad en un instante dado, y dicho instante marca un «punto de no retorno».

Una definición equivalente es

$$\sigma \in P \Rightarrow \forall i \geq 0 : \exists \beta : \sigma[..i]\beta \in P \tag{3}$$

Es decir, una traza respeta la propiedad de safety si en cada instante de tiempo existe una continuación posible que respeta la propiedad.

Liveness

La propiedad de liveness especifica que «siempre puede ocurrir» algo, donde el *algo* en cuestión se entiende como un evento deseable. Un conjunto de trazas $P \subseteq \Sigma^\omega$ es de liveness si cumple

$$\forall \sigma \in P : \exists \beta \in \Sigma^\omega : \sigma\beta \in P \tag{4}$$

Teorema. Toda propiedad $P \subseteq \Sigma^\omega$ puede escribirse como una intersección de una propiedad de safety y una propiedad de liveness.

Lógica Temporal Lineal (LTL)

Los operadores de LTL son:

$$
\begin{aligned}
\Box\psi &:= \text{ siempre, en el futuro, sucede } \psi \\
\Diamond\psi &:= \text{ siempre, en el futuro, sucede } \psi \\
\bigcirc\psi &:= \text{en el siguiente instante suecde } \psi \\
\psi \text{ U } \varphi &:= \psi \text{ sucede hasta que ocurra } \varphi \\
\psi \text{ WU } \varphi &:= \psi \text{ sucede siempre, o hasta que ocurra } \varphi
\end{aligned}
$$

Todo operador en realidad es una combinacón de los operadores básicos U y $\bigcirc$.

Sea $\mathcal{P}$ el conjunto de todas las proposiciones atómicas de LTA y sea $\mathcal{F}$ el conjunto de fórmulas. Toda $p \in \mathcal{P}$ pertenece a $\mathcal{F}$ por axioma. Si $\psi, \varphi \in \mathcal{F}$, entonces $\varphi \bowtie \psi \in \mathcal{F}$ para cualquier conector lógico (proposicional o de LTL) $\bowtie$, y $\neg\varphi, \bigcirc\varphi, \Diamond\varphi, \Box\varphi \in \mathcal{F}$.

Una teoría proposicional es un conjunto de fórmulas que incluye los axiomas de la lógica proposicional. Dada una teoría proposicional, todo modelo $\mathcal{A}$ de la teoría puede verse como un subconjunto $\mathcal{A} \subseteq \mathcal{P}$.

Más generalmente, diremos que φ se satiface en $\mathcal{A} \subseteq \mathcal{P}$ si $\mathcal{A} \vDash \varphi$, definiendo inductivamente

$$
\begin{aligned}
\mathcal{A} \vDash p &\iff p \in \mathcal{A} \\
\mathcal{A} \vDash \neg\varphi &\iff \mathcal{A} \nvDash \varphi \\
\mathcal{A} \vDash \varphi \wedge \psi &\iff \mathcal{A} \vDash \varphi \text{ y } \mathcal{A} \vDash \psi
\end{aligned}
$$

Siguiendo esta idea, decimos que un modelo de una fórmula de LTL es un subconjunto de

$$\left(2^{\mathcal{P}}\right)^{\omega}$$

Es decir, un modelo es un conjunto de secuencias infinitas de modelos de teorías proposicionales. Para decir que una fórmula de LTL φ se satisface en una traza $\sigma \in (2^{\mathcal{P}})^{\omega}$, decimos $\sigma \vDash \varphi$.

1 Model checking

El *modelo* de un sistema define un sistema de transiciones

$$M = (S, s_0, \rightarrow, v)$$

donde S son los estados posibles del sistema, s_0 el estado inicial, $\rightarrow$ es la relación de transición tal que si $s \in S$ existe un $s' \in S$ que cumple $s \rightarrow s'$, y $v : S \rightarrow 2^{\mathrm{PA}}$ es una función de valuación.

La función de valuación mapea estados a conjuntos de proposiciones atómicas. Si las proposiciones de PA expresan propiedades del sistema, la idea es que $v(S) \subseteq PA$ sean aquellas proposiciones que son verdaderas en el estado S.

Una ejecución de un modelo M es una función $\rho : \mathbb{N} \rightarrow S$ tal que

$$\rho(0) = s_0, \qquad \rho(i) \rightarrow \rho(i+1) \qquad \forall i \geq 0$$

El comportamiento de un modelo se define como

$$\mathcal{L}(M) = \left\{\sigma \in (2^{PA})^\omega \mid \exists \rho : \forall i \geq 0 : \sigma(i) = v\left(\rho\left(i\right)\right)\right\}$$

Es decir, es el conjunto de trazas que «se corresponden» con una ejecución de M, o que pueden producirse con una ejecución de M.

Hacer *model checking* es verificar si un sistema M cumple una propiedad lógica ϕ, algo que denotamos con $M \vDash \phi$. Decimos que

$$M \vDash \phi \iff \mathcal{L}(M) \subseteq \mathcal{L}(\phi)$$

donde recordamos que $\mathcal{L}(\phi)$ es el conjunto de todas las trazas donde ϕ es verdadera.

Autómatas de Büchi

Un autómata de Büchi es una estructura

$$\mathcal{A} = (\Sigma, S, \delta, s_0, A)$$

tal que Σ es un alfabeto finito, S un conjunto de estados con estado inicial s_0, $\delta : \Sigma \times S \rightarrow 2^S$ es una función de transición, y A es el conjunto de estados de aceptación.

Dado un automáta de Büchi $\mathcal{A}$ y una traza $\sigma = \sigma_0\sigma_1\sigma_2\ldots$, decimos que Σ es aceptada por $\mathcal{A}$ si existe $\rho \in \mathbb{N} \to S$ tal que

1. $\rho(0) = s_0$
2. $\rho(i+1) \in \delta\left(\sigma(i), \rho(i)\right)$
3. Existen estados de aceptación en A que se repiten infinitas veces en ρ; es decir, el conjunto $\{i \in \mathbb{N} \mid \rho(i) \in A\}$ es infinito.

Definimos $\mathcal{L}(A)$ como el conjunto de todas las trazas aceptadas por $\mathcal{A}$.

Los autómatas de Büchi aceptan exactamente todos lenguajes ω-regulares.

Notemos que si $M = (S, s_0, \to, v)$ es modelo de un programa, entonces M puede verse como el autómata de Büchi $\mathcal{A} = (\Sigma, S, \delta, s_0, S)$ tal que $\Sigma = 2^{\mathrm{PA}}$ y

$$s_j \in \delta(B, s_i) \iff s_i \to s_j \wedge B = v(s_i)$$

y todos los estados son de aceptación. Esto último se debe a que nos interesan todas las ejecuciones posibles del sistema.

Teorema. $\mathcal{L}(M) = \mathcal{L}(\mathcal{A})$.

Teorema. Para toda fórmula LTL ϕ, se puede construir un autómata de Büchi $\mathcal{A}$ tal que $\mathcal{L}(\mathcal{A}) = \mathcal{L}(\phi)$.

Teorema. Dados dos autómatas de Büchi $\mathcal{A}_1, \mathcal{A}_2<$ se puede construir un autómata $\mathcal{B} = \mathcal{A}_1 \cap \mathcal{A}_2$ tal que $\mathcal{L}(B) = \mathcal{L}(\mathcal{A}_1) = \mathcal{L}(\mathcal{A}_2)$.

Teorema. Lo mismo que el teorema anterior pero con el complemento.

Teorema. Existe un algoritmo que permite decidir si el lenguaje ω-regular aceptado por un autómata de Büchi es vacío o no.

2 P1

(3) Dar la descripción en FSP de los grafos en el práctico.

```
MOVIE = (ahead -> (right -> MOVIE | left -> STOP) ).
JOB = (arrive -> work -> leave -> JOB ).

GAME = ( tree-> lose -> GAME
     | {a, b} -> S
     ),
S = (win -> GAME).

FOURTICK = (tick -> tick -> tick -> tick -> STOP).


DOUBLE =
(
in[i:1..3] -> out[i] -> DOUBLE
).


PERSON =
(
  weekend -> SLEEP
| weekend -> SLEEPWORK
),

SLEEP = (sleep -> {play, shop} -> PERSON),
SLEEPWORK = (sleep -> work -> PERSON).
```

(4) Una variable almacena valores en el rango 0..N y soporta las acciones read y write. Modele la variable como un proceso, denominado VARIABLE, usando FSP. Para N = 2, compruebe que se pueden realizar las acciones dadas por la siguiente traza:

write.2 → read.2 → read.2 → write.1 → write.0 → read.0

```
const N = 2
range T = 0..N

VARIABLE = VARIABLE[0],
VARIABLE[u:T] = ( read[u]    -> VARIABLE[u]
                | write[v:T] -> VARIABLE[v]
                ).
```

(6) Considere un sensor que mide el nivel de agua de un tanque. El nivel, que inicialmente es 5, se mide en unidades de 0 a 9. El sensor emite una señal de bajo si el nivel es menor que 2 y una señal de alto si el nivel es mayor a 8. En cualquier otro caso, el sensor emite la señal normal. Modele el sensor como un proceso FSP.

El sensor estático se modela adecuadamente con:

```
const K = 9
range H = 0..K

SENSOR = SENSOR[5], // Nivel inicial en 5
SENSOR[j:H] =
(
   when (j < 2) bajo -> SENSOR[j]  // señal de bajo pero mantener el estado del
 | when (j > 8) alto -> SENSOR[j] // Same con señal de alto
 | when (j >= 2 && j <= 8) normal -> SENSOR[j] // Same con señal de normalidad
).
```

Sin embargo, ninguna de las acciones modifica el estado del sensor, y por lo tanto el grafo tiene un único nodo. Debemos agregar un mecanismo para modificar el estado del sensor.

```
const K = 9
range H = 0..K

SENSOR = SENSOR[5], // Nivel inicial en 5
SENSOR[j:H] =
(
   when (j < 2) bajo -> SENSOR[j]
 | when (j > 8) alto -> SENSOR[j]
 | when (j >= 2 && j <= 8) normal -> SENSOR[j]
    | setlevel[n:H] -> SENSOR[n]
).
```

(7) Una máquina expendedora de bebidas cobra 15c por una lata de gaseosas. La máquina acepta monedas de 5c, 10c, y 20c, y da vuelto. Modele la máquina como un proceso FSP.

```
const N = 35
range K = 0..N

EXP = EXP[0],
EXP[x:K] =
  ( when (x < 15)  in[5]  -> EXP[x+5]
  | when (x < 15)  in[10] -> EXP[x+10]
  | when (x < 15)  in[20] -> EXP[x+20]
  | when (x == 15) yield  -> EXP[0]
  | when (x > 15)  change[x-15] -> yield -> EXP[0]
  ).
```

(8) Considere una radio FM con tres controles. Uno es un interruptor on/off, que enciende el aparato o lo apaga. Los otros controles controlan la sintonización; éstos son dos botones scan y reset. Cuando la radio se enciende o el botón reset se apreta, la radio se sintoniza en la frecuencia más alta de la banda (es decir, 108Mhz). Cuando se presiona scan, la radio empieza a buscar una estación, bajando hacia la frecuencia más baja de la banda (88Mhz); se frena cuando encuentra una estación o cuando llega la banda más baja, y si se vuelve a apretar scan, continúa la búsqueda en forma descendente. Modele la radio como un proceso FSP.

```
const A = 88
const B = 108
range T = A..B

RADIO = (on -> ON[B]),
ON[i:T] = (
   off -> RADIO
 | station -> STATION[i]
 | reset -> ON[B]
 | when(i > A) scan -> ON[i-1]
),
STATION[i:T] = (
    when (i > A) scan -> ON[i-1]
 |  off -> RADIO
 |  reset -> ON[B]
).
```

(9) Muestre que S1 y S2, definidos de la siguiente manera:

```
P = (a -> b -> P).
Q = (c -> b -> Q).
||S1 = (P || Q).
S2 = (a -> c -> b -> S2 | c -> a -> b -> S2).
```

tienen el mismo comportamiento.

Comportamiento de `S1`: Como `a, c` no son compartidas, cualquiera puede ejecutarse primero. Si `a` se ejecuta primero, `b` no podrá ejecutarse y deberá ejecutarse `c`. El caso análogo ocurre si `c` se ejecuta primero. Por ende, las sub-trazas posibles son

```
a -> c -> b
c -> a -> b
```

que se repiten (tal vez intercalándose) indefinidamente. Es obvio por definición que este comportamiento es idéntico al de `S2`.

(10) Un museo permite que sus visitantes entren a través de la entrada este, y que abandonen el museo a través de la salida oeste. Los arribos y las partidas de personas son controladas por puertas giratorias en la entrada y en la salida. Además, el director del museo indica la habilitación de la entrada y salida de personas en el horario de apertura del museo, y cuando indica el horario de cierre sólo se permiten salidas del museo, pero no entradas. La habilitación e inhabilitación de las puertas giratorias las realiza un controlador. Modele, utilizando los procesos `EAST`, `WEST`, `CONTROL` y `DIRECTOR`, el comportamiento del museo.

Razonamiento. Las puertas este y oeste solo tienen una acción: entrar y salir, respectivamente. Ninguna de estas acciones modificia el estado del sistema.

El director tiene dos acciones: abrir y cerrar el museo. Ninguna modifica directamente el estado del sistema, sino que desencadenan (se sincronizan) con la acción correspondiente del controlador.

El controlador es la interfaz entre el director y el funcionamiento del sistema. La posibilidad de `entrar` así como el `abrir` y `cerrar` del museo dependen del controlador y por ende estas acciones serán compartidas. El controlador debe ofrecer `entrar`, `salir` y `cerrar` si el museo está abierto; `salir, abrir` si el museo está cerrado. Es claro entonces que el `CONTROLADOR` está formado por dos sub-procesos.

```
EAST = (entrar -> EAST).
WEST = (salir -> WEST).
DIRECTOR = (abrir -> cerrar -> DIRECTOR).

CONTROL = CERRADO,
CERRADO = (abrir -> ABIERTO
          | salir -> CERRADO),
ABIERTO = (entrar -> ABIERTO
          | salir -> ABIERTO
          | cerrar -> CERRADO).

||MUSEO = (EAST || WEST || DIRECTOR || CONTROL).
```

Notar que como `DIRECTOR` y `CONTROL` comparten abrir y cerrar, el `CONTROL` no podrá abrir sin que el director abra ni el `DIRECTOR` abrir sin que el control abra. Lo mismo vale para cerrar. Es decir, ambos procesos están coordinados. Es la forma en que modelamos que el director «ordena» la apertura y el cierre

de las puertas.

Como `CONTROL` solo ofrece la acción `entrar` en el proceso `ABIERTO`, la puerta `EAST` sólo tendrá esta acción habilitada en dicho proceso. Esto modela la inhabilitación de la puerta de entrada. La acción `salir`, por otra parte, siempre está disponible.

Figure 1: Image from the Museo folder

(11) Considere una célula de producción industrial, en la cual se procesan objetos, realizando un prensado de los mismos. Los objetos que se reciben son de cierto tipo A, y llegan a través de una cinta transportadora. Al llegar el primer elemento de la cinta a la posición de extracción, la cinta se para. Luego, un brazo mecánico levanta el elemento de la cinta y lo lleva a la prensa (la cual debe estar desocupada), para su prensado. Luego del prensado, otro brazo mecánico extrae el elemento de la prensa y lo deposita en la salida de la célula. Modele la célula de producción como un proceso FSP.

```
CINTA = (fill_cinta -> cinta_to_press -> CINTA).
PRENSA = (cinta_to_press -> prensar -> out -> PRENSA).

||CELL = (CINTA || PRENSA).
```

P2

Ejercicio 2. Considere el problema de lectores y escritores que desean acceder a un recurso común (por ej. una variable) donde los lectores sólo pueden leer de ese recurso mientras los escritores pueden leer y escribir. Además de las consideraciones usuales del problema de lectores-escritores, por una restricción de la administración del recurso, sólo se permite que K procesos lo estén accediendo simultáneamente. Suponga que en el sistema hay N lectores y M escritores (con N y M mayores a K). Modele el problema descripto en FSP, proveyendo además de las definiciones de los procesos correspondientes, el diagrama de estructura de su modelo.

Las acciones disponibles a los $N+M$ agentes son simplemente leer y escribir. Existen $N+M$ procesos activos (agentes).

No se regula el acceso de los lectores, pero un escritor solo puede acceder si no hay nadie más en el recurso. De otro modo, escribiría sobre lo que otro lee o sobre lo que otro escribe. En resumen, en el caso de los escritores, debemos garantizar exclusión mutua.

Para lograr todo, basta definir un semáforo que regule la cantidad de agentes en el recurso, y hacer que los lectores y los escritores se relacionen de forma diferente con el semáforo. Para

Para lograr todo, definimos un proceso pasivos que es como un semáforo alterado. Este semáforo modificado funciona tal y como un semáforo, con dos excepciones: (a) tiene un límite superior K, (b) cuando está en cero se habilitan las acciones `acquire -> release` (es decir, cuando está en cero, se puede comportar como un `lock`).

Los lectores solo harán *up* y *down* sobre el semáforo, no se preocupan por nada del lock. Los escritores harán `acquire`, una acción que se bloqueará hasta que el semáforo esté en cero (i.e. hasta que no haya ningún escritor y ningún lector).

Notar que los escenarios posibles son: hay k lectores, con $0 \leq k \leq \max$ (regulado por el semáforo); o hay 1 escritor (regulado por «el lock del semáforo»). La solución está en la siguiente página.

```
const N = 3
const M = 3
const K = 2

SEMLOCK(V=0) = SEM[V],
SEM[x:0..(N+M)] = (
   when (x < K) up -> SEM[x+1]
 | when (x > 0) down -> SEM[x-1]
 | when (x == 0) acquire -> release -> SEM[0]).

READER = (semlock.up -> read -> semlock.down -> READER).
WRITER = (semlock.acquire -> write -> semlock.release -> WRITER).


||MODEL = (
 r[1..N]:READER ||
 w[1..M]:WRITER ||
 {r[1..N], w[1..M]}::semlock:SEMLOCK
).
```

Notamos que el alfabeto definido `SEMLOCK` tiene un par de redundancias; en particular, las acciones `rj.acquire, rj.release` con $j = 1 \ldots N$ no van a tener relevancia en la composición final (los escritores no hacen release ni acquire).

Ejercicio 3. Una computadora central conectada a terminales remotas a través de enlaces de comunicación se utiliza para realizar reservas de asientos en un teatro. Los empleados del teatro que realizan las reservas pueden mostrar el estado actual de reservas en las pantallas de las terminales. Para poder reservar un asiento, el cliente elije el número, y un empleado accede a través de la terminal para saber si el mismo está reservado o no. Si no lo está, realiza la reserva correspondiente. Construya un modelo FSP para este problema, asegurando que no puede reservarse más de una vez el mismo asiento en el teatro.

Tenemos empleados como agentes externos al sistema que pueden interactuar con cualquiera de las k terminales. El sistema no sabe nada del cliente: solo recibe a través de T_j un número x y responde si está habilitado o no. El problema central es: ¿cómo «llevar la cuenta» de qué asiento está libre o no?

Para esto, modelamos los N asientos como parte del sistema y modelamos que la computadora central está en relación con ellos.

(11) Muestre que los siguientes procesos son (fuertemente) bisimilares:

```
P = a -> (b -> c -> P)
```

$$P = (a \to (b \to c \to P \quad | \quad b \to c \to P)) \,.$$

$$Q = (a \to b \to R \quad | \quad a \to b \to R) \,, \qquad R = (c \to Q) \,.$$

3 P3

(3) Demuestre que la intersección de propiedades de safety es una propiedad de safety.

Sean $P_1, P_2 \subseteq \Sigma^\omega$ propiedades de safety. Por def. tenemos que

$$\alpha \in P_i \Rightarrow \forall i \geq 0 : \exists \beta \in \Sigma^\omega : \alpha[..i]\beta \in P_i$$

para $i = 1, 2$. Sea $P = P_1 \cap P_2$ y $\alpha \in P$. Entonces $\alpha \in P_1$ y $\alpha \in P_2$. Por lo tanto,

$$\forall i \geq 0 : \exists \beta \in \Sigma^\omega : \alpha[..i]\beta \in P_i$$

para $i = 1$ y para $i = 2$. Sea $\beta_0 \in \Sigma^\omega$ un elemento arbitrario tal que $\alpha[..i]\beta_0 \in P_i$ para $i = 1, 2$ para todo $i \geq 0$. Por def.

$$\alpha[..i]\beta_0 \in P_1 \wedge \alpha[..i]\beta_1 \in P_2 \Rightarrow \alpha[..i]\beta_0 \in P$$

$$\therefore \ \forall i \geq 0 : \exists \beta \in \Sigma^\omega : \alpha[..i]\beta \in P$$

$\therefore$ P es una propiedad de safety.

(4) ¿Es cierto que la intersección también preserva las propiedades liveness?

Sean P_1, P_2 propiedades de liveness y sea $P := P_1 \cap P_2$. Entonces

$$\forall \alpha \in P_i : \exists \beta \in \Sigma^\omega : \sigma\beta \in P_i, \qquad i = 1, 2$$

Sea $\alpha \in P$, i.e. $\alpha \in P_1$ y $\alpha \in P_2$. Consideremos como contraejemplo el caso en que el $\beta_0 \in \Sigma^\omega$ tal que $\alpha\beta_0 \in P_1$ es único, y el β_1 tal que $\alpha\beta_1 \in P_2$ es único, y $\beta_0 \neq \beta_1$. Se sigue que $\alpha\beta_0 \notin P_2$ y $\alpha\beta_1 \notin P_1$. Si este fuera el caso, se tendría $\alpha\beta_0, \alpha\beta_1 \notin P$. Si hubiera otra traza $\hat{\beta}$ tal que $\alpha\hat{\beta} \in P$, se violaría la unicidad de β_1 y β_2. Por lo tanto, en este caso, no existe ningún $\beta \in \Sigma^\omega$ tal que $\alpha\beta \in P$.

$\therefore$ P no es una propiedad de liveness.

(5) Encuentre alguna propiedad que sea simultáneamente de safety y de liveness.

Es muy sencillo observar que dado un alfabeto Σ, el lenguaje ω-regular Σ^ω es una propiedad de safety y es una propiedad de liveness. La pregunta es si existe una propiedad que sea simultáneamente safety y liveness más allá de esta, que es trivial. Demostraremos que esto es imposible.

Sea $P \subset \Sigma^\omega$ una propiedad de safety y de liveness. Como $P \neq \Sigma^\omega$, se sigue que $\overline{P} \neq \emptyset$, donde el complemento se toma respecto al universo Σ^ω. En otras palabras, existen trazas que llegan a un «punto de no retorno».

Por hipótesis, P satisface dos propiedades simultáneamente:

1. Si $\alpha \notin P$, existe un prefijo $\alpha[..i]$, con $i \geq 0$, tal que $\beta \notin P$ para todo $\beta \in \Sigma^\omega$.
2. Dada cualquier palabra $\psi \in \Sigma^*$, existe un $\varphi \in \Sigma^\omega$ tal que $\psi\varphi \in P$. En otras palabras, cualquier tramo inicial puede llegar a formar una traza en P.

Sea $\alpha \in \overline{P}$. Como P satisface safety, se cumple que α tiene un prefijo malo $\alpha[..i]$, $i \geq 0$, tal que $\alpha[..i]\beta \notin P$ para toda traza β posible. Pero $\alpha[..i] \in \Sigma^*$. Por liveness, existe una traza $\varphi \in \Sigma^\omega$ tal que $\alpha[..]\varphi \in P$. Pero esto contradice que $\alpha[..i]$ es un prefijo malo. La contradicción viene de asumir que $\overline{P} \neq \emptyset$.

$\therefore$ Sólo Σ^ω cumple simultáneamente liveness y safety.

(†) Notar que ya hemos realizado acá el ejercicio **(6)**.

(7) Demustre o refute que el complemento de una propiedad de safety es una propiedad de liveness.

Ya vimos que Σ^ω es de safety y de liveness. Si el enunciado fuera cierto, $\overline{\Sigma^\omega} = \emptyset$ sería de liveness. Esto querría decir que para toda $\alpha \in \Sigma^\omega$, existe una traza no vacía e infinita β tal que $\alpha\beta \in \emptyset$, lo cual es absurdo.

$\therefore$ El enunciado es falso.

(8) Considere el alfabeto $\Sigma = \{a, b\}$. Determine si las siguientes propiedades son de *safety*, *liveness*, ambas, o ninguna. Justifique la respuesta:

i. a^*b^ω	*iii.* $a^*b^+a^\omega$	*v.* $(ab)^\omega$
ii. $(b+a)^+b^\omega$	*iv.* $(a+b)^*(a^\omega+b^\omega)$	*vi.* $(ab)^*a^\omega$

(i) Este lenguaje ω-regular consiste de las palabras que empiezan con una cantidad arbitraria, posiblemente nula, de as, seguida de una traza infinita de bs.

(Safety.) Sea α una palabra *fuera* del lenguaje. Una forma posible de α es ser una cadena infinita de as. Pero si α tiene esta forma, todo prefijo de α es una cadena finita de as, a la cual puede agregarse una cadena infinita de bs para «corregir» la traza y dar con una traza buena. Es decir, no se cumple que toda traza fuera del lenguaje tiene un prefijo malo. $\therefore$ No es de safety.

(Liveness.) Sea $\varphi = aba \in \Sigma^*$. Claramente, no existe ninguna traza $\psi \in \Sigma^\omega$ tal que $\varphi\psi$ pertenezca al lenguaje. $\therefore$ No es de liveness.

(ii) El lenguaje tiene una combinación arbitraria de as y bs al empezar, posiblemente nula, pero eventualmente «se queda» en b y la repite indefinidamente.

(Safety) Si α no pertenece al lenguaje, debe cumplirse que una de las siguientes: o bien α tiene una repetición infinita de as, o es una combinación infinita de as y bs, sin nunca «quedarse» en b. De esto se sigue que para todo $i \geq 0$, $\alpha[i]$ es o bien a o bien b. Pero de esto se sigue que $\alpha[..i]b^\omega$ es una traza en lenguaje. Es decir, ningún i satisface que $\alpha[..i]$ sea un prefijo malo. $\therefore$ No es de safety.

(Liveness) Sea $\varphi \in \Sigma^*$. Si φ es una combinación de as y bs en orden y cardinalidad arbitrarios. Pero entonces es claro que φb^ω pertenece al lenguaje. (Notar que esto vale incluso si $\varphi = \epsilon$.) $\therefore$ Es de liveness.

(iii)

(Safety) Sea α una palabra fuera del lenguaje. Es concebible que α sea de la forma b^ω. Pero entonces todo prefijo de α es de la forma b^n para n natural. Pero entonces αa^ω pertenece al lenguaje. Es decir que b^n no es un prefijo malo. Como no todas las palabras fuera del lenguaje tienen un prefijo malo, la propiedad no es de safety.

(Liveness) Sea $\alpha = bab \in \Sigma^*$. No importa qué traza infinita añadamos, no pertenece al lenguaje. No es de liveness.

(iv) (Safety.) Suponga que α no pertenece al lenguaje. Entonces α es una traza que combina as y bs indefinidamente, sin nunca «quedarse» en una o la otra. Obviamente no tiene prefijo malo, porque $\alpha[..i]z^\omega$, con $z \in \Sigma$, pertenece al lenguaje para toda $i \geq 0$.

(Liveness.) Por def. toda palabra del lenguaje empieza con una $\varphi \in \Sigma^*$; o bien, toda $\varphi \in \Sigma^*$ es tramo inicial de una traza del lenguaje. De esto se sigue liveness trivialmente.

(v)

(Safety.) Toda palabra fuera del lenguaje es o bien una palabra que empieza con b o bien una palabra que empieza con a pero no es exactamente alternante, i.e. tiene una a tras la cual no hay una b, o una b tras la cual no hay una a.

Si no es exactamente alternante, existe un índice i tal que $\alpha[i] = \alpha[i+1]$. Tomando $\alpha[..(i+1)]$, es claro que $\alpha[..(i+1)]\beta$, con $\beta \in \Sigma^\omega$, no pertenece al lenguaje. Es decir, existe un prefijo malo.

Si α empieza con b, entonces $\alpha[..0]$ es el prefijo malo.

$\therefore$ Es de safety.

Sea $\psi = aa$. No existe $\beta \in \Sigma^\omega$ tal que $\psi\beta$ pertenezca al lenguaje. No es de liveness.

(vi) $(ab)^\omega$ no pertenece al lenguaje pero no tiene prefijo malo, dado que $(ab^\omega)[..(2i+1)]a^\omega$ pertenece al lenguaje para toda i. No es de safety.

Si $\psi = abb$, no existe traza infinita que añadida a ψ pertenezca al lenguaje. No es de liveness.

(9) Para cada propiedad P del Ejercicio 8 que no sea de safety extiéndala agregando el menor conjunto de trazas de manera que lo sea. Use expresiones ω-regulares. Justifique su respuesta

$(i) P = a^* b^\omega$.

Hay dos tipos de palabra fuera de P: las que, tras una aparición de b, tienen alguna aparición de a, y las que son una secuencia infinita de as.

Las primeras tienen prefijo malo (trivial), pero las segundas no, porque si α es una secuencia infinita de as, $\alpha[..i]b^\omega \in P$ para todo $i \geq 0$.

Por lo tanto, extendemos el lenguaje y definimos $P' = a^* b^\omega + a^\omega$. La idea es que ahora P' contiene los elementos fuera del lenguaje P que no tenían prefijo malo. Ahora las únicas palabras fuera de P son las que tienen alguna a después de la primera aparición de b, y la propiedad es de safety. Notamos que $P \subset P'$, que es lo deseado.

$(ii) P = (b + a)^+ b^\omega$

En el ejercicio anterior probamos que esta propiedad era de liveness. La clausura de safety de una propiedad de liveness es Σ^ω. $\therefore$ Debemos extender el lenguaje a toda Σ^ω.

> **Demostración del enunciado anterior.** Sea P una propiedad de liveness. Todo prefijo $\alpha[..i]$, con $i \in \mathbb{N}$ y $\alpha \in \Sigma^\omega$, es una palabra de Σ^*. Por lo tanto, existe un $\beta \in \Sigma^\omega$ tal que $\alpha[..i]\beta \in P$. Como esto vale para $\alpha \in \Sigma^\omega$, debe valer en particular para $\alpha \in \overline{P}$.
>
> $\therefore$ Toda palabra fuera de P carece de prefijo malo.
>
> La clausura de safety de P debe incorporar a P todas las palabras fuera de P que no tienen prefijo malo. Pero hemos visto que esto es todas las palabras en $\overline{P}$.
>
> $\therefore$ La clausura de safety de P es $P \cup \overline{P} = \Sigma^\omega$.

$(iii) P = a^* b^+ a^\omega$

Sea $\alpha \notin P$. Entonces α es de alguno de los siguientes lenguajes:

- Trazas infinitas de a: $\alpha \in a^\omega$
- Repeticiones infinitas de bs, tal vez con una secuencia inicial de as: $\alpha \in a^* b^\omega$
- Palabras que respetan la estructura inicial de P, pero no se «quedan» en una repetición infinita de as : $\alpha \in a^* b^+ a^+ b^+ \beta$ con $\beta \in \Sigma^\omega$ cualquiera.

La primera y la segunda forma no tienen prefijo malo (trivial). Pero la tercera sí: el prefijo malo es $a^n b^m a^k b$.

Proponemos $P' = a^* b^+ a^\omega + a^\omega + a^* b^\omega$

$(iv)(a+b)^*(a^\omega + b^\omega)$

Me aburrí.

$(vi)(ab)^* a^\omega$

Me aburrí.

(10) Para cada P del ejercicio **(8)** que no sea ni de safety ni de liveness, dé una propiedad de safety S y una de liveness L tal que $P = S \cap L$.

$(i) a^* b^\omega$

$$L = (a+b)^* b^\omega, \qquad S = a^* b^\omega + a^\omega$$

Que L es de liveness se sigue de que pra toda $\alpha \in \Sigma^*$, $\alpha b^\omega \in L$.

Que S es de safety se sigue que si $\alpha \notin S$ entonces S contiene alguna transición de b a a; i.e. un índice tal que $\alpha[i] = b$ y $\alpha[i+1] = a$. (Alternativamente, S es la clausura de safety dada en el ejercicio anterior.)

Como las palabras de L terminan con trazas infinitas de bs y las de S con trazas infinitas de o bien as o bien bs, la intersección $L \cap S$ tiene palabras que terminan con trazas infinitas de b. Y como las de L empiezan con una combinación arbitraria de as o bs (tal vez vacía), y las de S con una secuencia de as)(tal vez vacía), las de $L \cap S$ empiezan con una secuencia de as (tal vez vacía). Es decir,

$$L \cap S = a^* b^\omega$$

(iii) $a^* b^+ a^\omega$

Sea $S = a^* b^+ a^\omega + a^\omega + a^* b^\omega$.

$(vi) (ab)^\omega$

P4

(2) Analizar y justificar:

1. $\Box\Box\varphi \to \Box\varphi$
2. $\Box(\varphi \wedge \psi) \to \Box\varphi \wedge \Box\psi$
3. $\Diamond(\varphi \vee \psi) \to \Diamond\varphi \vee \Diamond\psi$

(1) Asuma que $\sigma \vDash \Box\Box\varphi$. Entonces $\forall i \geq 0$ se cumple $\sigma[i..] \vDash \Box\varphi$. En particular, para $i = 0$, se cumple $\sigma[0..] = \sigma \vDash \Box\varphi$.

(2) Asuma que $\sigma \vDash \Box(\varphi \wedge \psi)$. Entonces $\forall i \geq 0$ se cumple $\sigma[i..] \vDash (\varphi \wedge \psi)$. Es decir que

$$\sigma[i..] \vDash \varphi \text{ y } \sigma[i..] \vDash \psi, \qquad \forall i \geq 0 \tag{1}$$

Por def. si para todo $i \geq 0$ se tiene $\sigma[i..] \vDash \phi$ entonces $\sigma \vDash \Box\phi$. Es decir que de la eq. (1) se sigue

$$\sigma \vDash \Box\varphi \text{ y } \sigma \vDash \Box\psi$$

Por ende $\sigma \vDash \Box\varphi \wedge \Box\psi$. ■

(3) Asuma que $\sigma \vDash \Diamond(\varphi \vee \psi)$. Es decir que $\exists i \geq 0$ tal que $\sigma[i..] \vDash (\varphi \vee \psi)$. Tomemos i como fijo y asumamos que $\sigma[i..] \vDash \varphi$ (la prueba si $\sigma[i..] \vDash \psi$ es análoga.) Como entonces se garantiza la existencia de $i \geq 0$ tal que $\sigma[i..] \vDash \varphi$, se tiene $\sigma \vDash \Diamond\varphi$. Pero entonces

$$\sigma \vDash \Diamond\varphi \;\vee\; \sigma \vDash \Diamond\psi$$

Pero entonces $\sigma \vDash \Diamond\varphi \vee \Diamond\psi$.

(3) Demuestre o refute la validez de

$$\Box(p \wedge \Diamond q) \iff \Box(p\mathrm{U}q)$$

Veamos que

$$\Box(p \wedge \Diamond q) \equiv \Box p \wedge \Box\Diamond q$$

Considere la siguiente traza σ, donde damos los valores de p y q en los sucesivos instantes:

$$\begin{array}{l} p \mid \mathbf{T} \mid \mathbf{F} \mid \mathbf{T} \mid \mathbf{F} \mid \ldots \\ q \mid \mathbf{F} \mid \mathbf{T} \mid \mathbf{F} \mid \mathbf{T} \mid \ldots \end{array}$$

Es obvio que $\sigma \not\models \Box p$. Probaremos que $\sigma \models \Box(p\mathrm{U}q)$.

Consideremos primero la traza σ completa. Tomando $j = 1$, vemos que se cumple $\sigma[j..] \models q$. Es fácil ver que para todo $0 \leq k < j$ se cumple $\sigma[k..] \models p$, porque el único valor en dicho rango es $k = 0$ y claramente $\sigma \models p$.

$\therefore \sigma \models p\mathrm{U}q$.

Ahora consideramos la traza $\sigma[1..]$, i.e. empezando justo en el segundo instante. Con $j = 0$ vemos que se cumple $(\sigma[1..])[0..] \models q$. Ahora deseamos probar que para todo $0 \leq k < j$ se cumple $(\sigma[1..])[k..] \models p$. Pero como $j = 0$, el intervalo $0 \leq k < j$ es vacío. Y toda cuantificación universal sobre un conjunto vacío es trivialmente verdadera.

$\therefore \sigma[1..] \models p\mathrm{U}q$

Como $\sigma = \sigma[2..] = \sigma[4..] = \ldots$ y $\sigma[1..] = \sigma[3..] = \sigma[5..]$, nuestra demostración se extiende a los naturales y hemos probado que $\forall i \geq 0 : \sigma[i..] \models p\mathrm{U}q$.

$\therefore \sigma \models \Box(p\mathrm{U}q)$

Pero la traza σ *no* cumple $\Box p$ y por ende no cumple $\Box(p \wedge \Diamond q)$.

$$\therefore \text{No es cierto que} \quad \Box(p \wedge \Diamond q) \iff \Box(p\mathrm{U}q)$$

(4) Demustre que strong fairness implica weak fairness.

Weak fairness sobre un proceso x significa que siempre que si el proceso está siempre Habilitado, entonces eventualmente se Ejecutará:

$$\Diamond\Box H \to \Box\Diamond E$$

Strong fairness indica que si un evento está frecuentemente habilidado, entonces finalmente se ejecutará:

$$\Box\Diamond H \to \Box\Diamond E$$

(Por contradicción.) Asuma que tenemos strong fairness pero no weak fairness. Entonces las siguientes tres fórmulas deben ser verdaderas:

$$\Box\Diamond H \to \Box\Diamond E, \qquad \Diamond\Box H, \qquad \neg\Box\Diamond E$$

Como la tercera fórmula es la negación del consecuente de la primera, debe cumplirse además

$$\neg\Box\Diamond H$$

Luego la siguiente fórmula es verdadera:

$$\neg\Box\Diamond H \wedge \Diamond\Box H \tag{2}$$

Pero $\neg\Box\Diamond H \equiv \Diamond(\neg\Diamond H)$ por el axioma que establece que $\neg\Box x \equiv \Diamond\neg x$. Es decir que la fórmula (2) puede escribirse como

$$\Diamond(\neg\Diamond H) \wedge \Diamond(\Box H) \tag{3}$$

La fórmula izquierda dice: a partir de cierto momento, H ya nunca más volverá a pasar. La fórmula derecha dice: a partir de cierto momento, H siempre pasará. Obviamente son contradictorias, pero debemos demostrarlo formalmente.

La fórmula izquierda garantiza la existencia de un j tal que $\sigma[j..] \vDash \neg\Diamond H$. Pero entonces $\sigma[j..] \nvDash \Diamond H$. Pero entonces no existe ningún $k \geq j$ tal que $\sigma[k..] \vDash H$.

La fórmula derecha garantiza la existencia de un j' tal que $\sigma[j'..] \vDash \Box H$. Es decir que $\forall i \geq j'$ se cumple $\sigma[i..] \vDash H$.

Sea $i = \max(j, j')$ de manera que $i \geq j, i \geq j'$. Por el análisis de la fórmula izquierda, debemos tener que *no* se cumple $\sigma[i..] \vDash H$. Por el análisis de la fórmula derecha, debemos tener que $\sigma[i..] \vDash H$ *sí* se cumple.

La contradicción se sigue de asumir que strong fairness no implica weak fairness.

(5) Intente expresar en lógica temporal lineal las siguientes propiedades:

1. "Si ϕ es cierto durante la ejecución del programa, eventualmente ocurrirá ψ"
2. "No es posible que ϕ y ψ ocurran simultáneamente durante la ejecución del programa"
3. "Cada vez que ϕ sea cierto, ψ también es"
4. "ϕ no deja de ocurrir, al menos hasta que ψ ocurra"

(1) $\Box(\phi \Rightarrow \Diamond\psi)$

(2) $\Box\left(\neg\left(\phi \wedge \psi\right)\right)$

(3) $\Box(\phi \rightarrow \psi)$

(4) $\phi \mathrm{WU} \psi$

(7) Dadas dos fórmulas LTL ϕ y ψ, sea $\phi \ \mathcal{Z} \ \psi$ el lenguaje conteniendo todas las secuencias $\rho \in (2^{PA})^\omega$ tales que, para dos posiciones distintas en las cuales ϕ es válida, ψ es válida en una tercera posición entre medio de las otras dos. Dé una definición de $\phi \ \mathcal{Z} \ \psi$ usando los operadores de lógica LTL.

Hay dos interpreaciones del problema. Una nos dice que, entre dos ocurrencias de ϕ, ψ tiene que valer en una (pero tal vez más) posiciones intermedias. La otra es que ψ tiene que valer en *exactamente* una posición intermedia.

(a) Con la primera interpretación, proponemos

$$\Box\Big[(\phi \wedge \bigcirc\Diamond\phi) \Rightarrow \bigcirc\,(\neg\phi \ \mathrm{U} \ (\psi \wedge \neg\phi))\Big]$$

(b) En la segunda interpretación, pedimos que la ocurrencia intermedia de ψ sea única. La propiedad se vuelve

$$\Box\Big[(\phi \wedge \bigcirc\Diamond\phi) \Rightarrow \bigcirc\Big[(\neg\phi \wedge \neg\psi) \ \mathrm{U} \ \Big(\psi \wedge \neg\phi \wedge \bigcirc(\neg\psi \ \mathrm{U} \ \phi)\Big)\Big]\Big]$$

Ahora, una vez que se cumple ϕ, pedimos que ϕ no se cumpla al menos hasta que suceda lo siguiente: ψ ha valido, y a partir del instante siguiente en que ψ ha avalido, no vuelve a valer hasta que vuelve a ocurrir ϕ, donde la ocurrencia de ϕ debe cumplirse necesariamente.

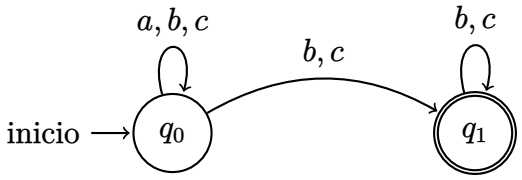
Independientemente de si seguimos la interpretación a o b, sea ξ la fórmula dada. Entonces el conjunto $\mathcal{Z}$ se define como

$$\mathcal{Z} := \Big\{\sigma \in (2^{\mathrm{PA}})^\omega \mid \sigma \vDash \xi\Big\}$$

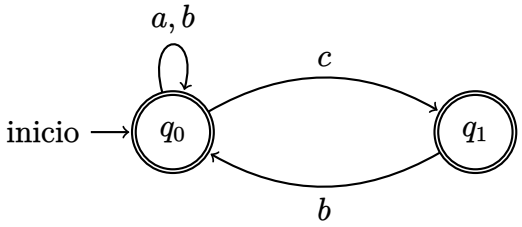
(8) Defina autómatas de Büchi para los siguientes lenguajes sobre el alfabeto $\{a, b, c\}$:

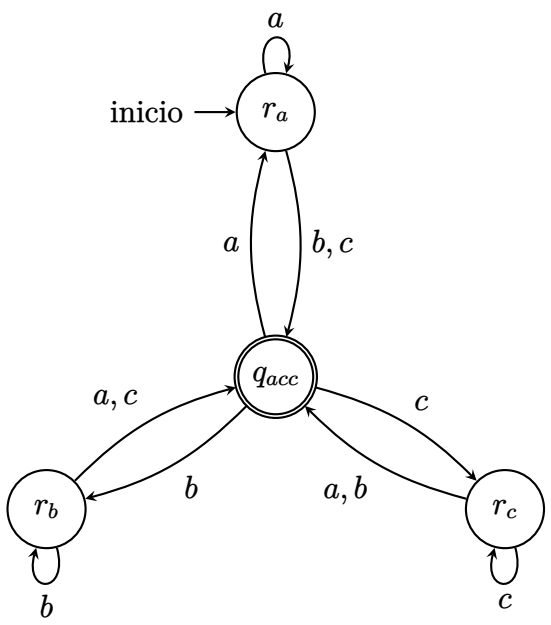
1. conjunto de cadenas infinitas con una cantidad finita de a,
2. conjunto de cadenas infinitas en que cada ocurrencia de c viene inmediatamente seguida de una ocurrencia de b,
3. conjunto de cadenas infinitas que no terminen en una secuencia infinita de a ni en una de b ni en una de c,
4. conjunto de cadenas infinitas con cantidades finitas de a, b y c.

(1) La idea es permitir que b y c mapeen *opcionalmente* al estado final y produzcan una permanencia en el estado final. Del estado final no debe haber ninguna transicón que involucre a a: de este modo, si en alguna traza la aparición de b o c llevó al estado final, una ocurrencia de a en dicha traza la detiene por completo, y nos asegura que el estado final no ocurre infinitas veces (no es aceptado).



(2) Dos estados: q_0 es cualquier cosa excepto la ocurrencia de una c, q_1 es «ha ocurrido una c». De q_1 solo se sale si ocurre una b. Toda cadena en que no ocurre una b justo después de una c se queda sin transiciones disponibles y se detiene. Al hacerse finita, no producirá infinitos estados de aceptación.




a
inicio
r_a
a
b, c
q_{acc}
a, c
c
r_b
b
a, b
r_c
b
c

(9) Probar que los autómatas dados aceptan el mismo lenguaje ω-regular.

A nivel intuitivo, es claro que el autómata izquierdo acepta el conjunto de palabras que contienen alguna a. El segundo también, pero agrega algunas transiciones molestas. Probemos formalmente que

$$\mathcal{L}(\mathcal{A}_1) = \{\sigma \in \Sigma^\omega : \exists i \geq 0 : \sigma(i) = a\} =: \mathcal{Z}$$

Sea $\sigma \in \mathcal{Z}$ arbitraria. Por def. existe un mínimo i tal que $\sigma(i) = a$. Definimos entonces

$$\rho(n) = s_0, \forall n \leq i, \qquad \rho(n) = s_1, \forall n > i$$

Probemos que esta ρ satisface las condiciones deseadas. En primer lugar, $\rho(0) = s_0$. Ahora hacemos un análisis por por casos.

(Caso $n <$) En este caso tenemos

$$\rho(n+1) = s_0$$

y por ser $n < i$ sabemos que $\sigma(n) = b, \rho(n) = s_0$. Pero $\delta(b, s_0) = \{s_0\}$. Es decir, $\forall n < i - 1$, se cumple $\rho(n+1) \in \delta(\sigma(n), \rho(n))$.

(**Caso** $n > i$) En este caso tenemos

$$\rho(n+1) = s_1$$

y por ser $n \geq i$ no podemos afirmar nada de $\sigma(n)$; i.e. $\sigma(n) \in \{a, b\}$. Pero $\delta(x, s_1) = \{s_1\}$ para $x = a$ y para $x = b$. Es decir que en cualquier caso se cumple $\rho(n+1) \in \delta(\sigma(n), \rho(n)$.

(**Caso** $n = i$). Este caso representa la transición de estado. Sabemos que

$$\rho(n) = s_0, \qquad \rho(n+1) = s_1$$

y que $\sigma(n) = a$. Pero $\delta(a, s_0) = \{s_1\}$. Es decir que $\rho(n+1) \in \delta(\sigma(n), \rho(n))$.

Vimos que en los tres casos se cumple la propiedad deseada. Del hecho de $\rho(n) = s_1$ para todo $n > i$ se sigue que $|\{i \in \mathbb{N} : \rho(i) \text{ es de aceptacion }\}| = \infty$.

$\therefore$ Todas las palabras en $\mathcal{Z}$ son aceptadas por $\mathcal{A}1$.

Probar que toda palabra fuera de $\mathcal{Z}$ no es aceptada es una pavada y lo salteamos.

$\therefore \mathcal{L}(\mathcal{A}_1) = \mathcal{Z}$. ■

Haciendo una prueba similar sobre el segundo autómata $\mathcal{A}_2$ no es difícil ni tan distinto a la prueba dada para $\mathcal{A}_1$.

Within each TMS session,

$$\rho = \frac{\text{Paired-pulse MEP}}{\text{Mean Single-pulse MEP}}$$

You are correct in claiming that we compute separately ρ for ISI = 4ms and ISI = 20ms.

Rationale for logarithmic transformation. For ISI 20ms, a single value ρ_{ICF} tells you the amount of facilitation that occurred. For ISI = 4 ms, ρ_{ICI} the amount of inhibition. So it would seem that we can proxy E/I balance simply as

$$Z = \frac{\rho_{\text{ICF}}}{\rho_{\text{ICI}}}$$

But there is a problem; ρ_{ICF} is in a different scale than $\rho(\text{ICF})$. Ideally, if a facilitatory pulse was x times larger than the test pulses, and an inhibitory pulse was x times smaller, these two values of ρ would be at the same distance from 1, since 1 means «the value was no different from a test pulse». But this is not true. For instance, for a response twice as large as a test pulse ρ_{ICF} will be two, but for a response half as large as a test pulse ρ_{ICI} will be 0.5. Their distance from one is not the same.

In other words, facilitation effects «move ρ above 1» at a different rate than inhibition effects «move ρ below 1». This is a silly way to put it but it might help.

What does this mean for E/I balance? Assume $\rho_{\text{ICF}} = 2$, i.e. facilitatory effect made pulses with twice the amplitude of test pulses. Assume $\rho_{\text{ICI}} = 0.5$, i.e. inhibitory effect made pulses with half the amplitude of test pulses. This is a pretty balanced state (same degree of inhibition and facilitation), but

$$\frac{\rho_{\text{ICF}}}{\rho_{\text{ICI}}} = \frac{2}{0.5} = 4$$

which makes no sense, since this indicates strong excitatory dominance!

It is in order to fix this that we take the logarithm. The logartihm corrects this problem. It's not worth going into why, but simply note that, in the same example as before,

$\log(2) \approx 0.30, \log(0.5) \approx -0.30$. So now it *is* the case that facilitatory and inhibitory effects of the same degree (twice as much, half as much) are at the same distance from the point of reference (now zero).

Then we simply define

$$Z := \log(\rho_{\mathrm{ICF}}) - \log(\rho_{\mathrm{ICI}})$$